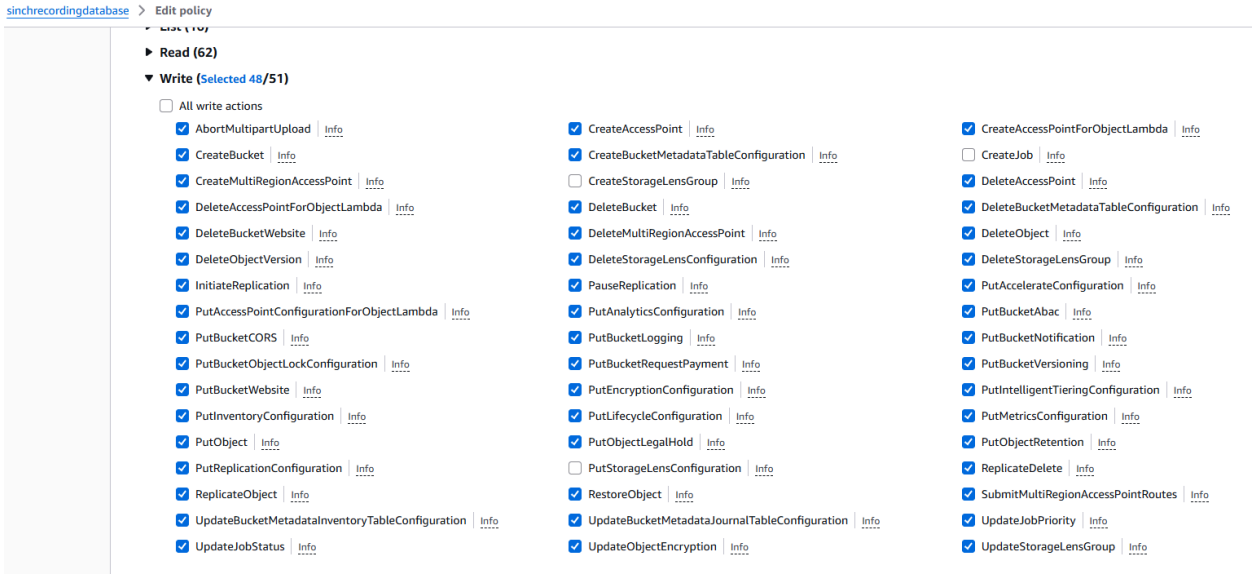
AWS S3 Hosted Voice recording

In this project, I created 2 IAM accounts. One with only write access for files to be uploaded to the S3 bucket, and another with only read permissions for the account to be able to access and download the recorded files. I also wrote a Lambda function that would auto deploy a new s3 folder if a recording was uploaded from a new hosted phone account to keep files organized.

ADDING IAM USERS

1. I created an IAM sinchrecordingdatabase user and give them inline s3 write permission

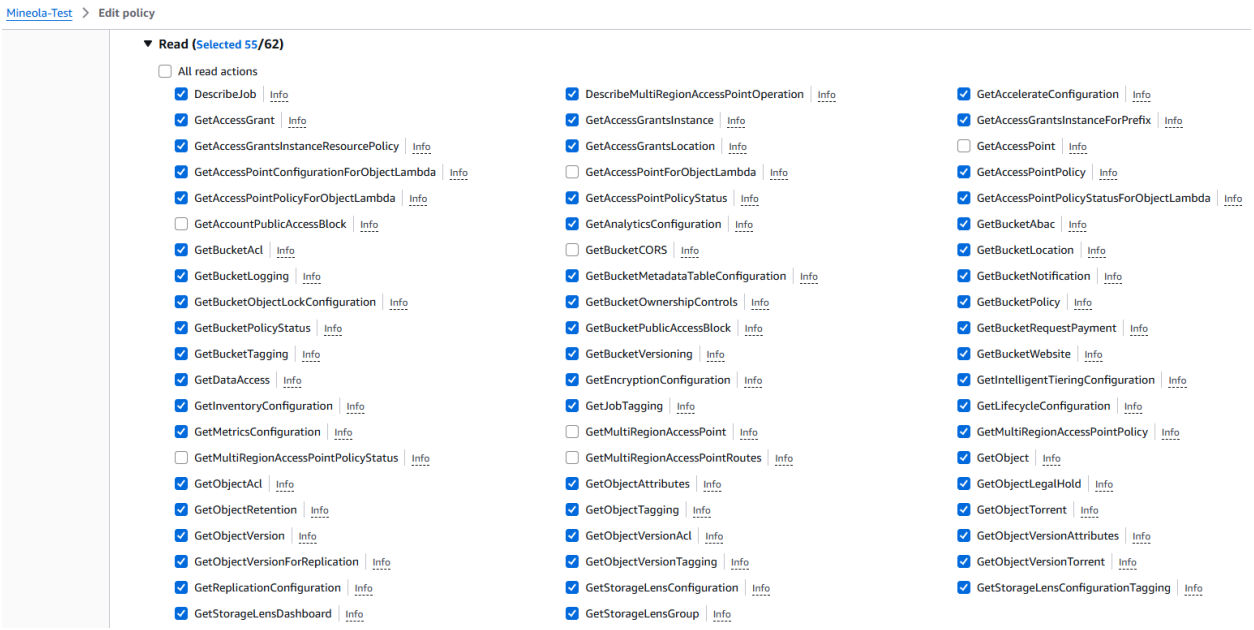


This allowed them to record hosted voice audio and upload to the designated s3 buckets.

I also generated an access key and security key to be used by Sinch to automatically upload the content to the folders.

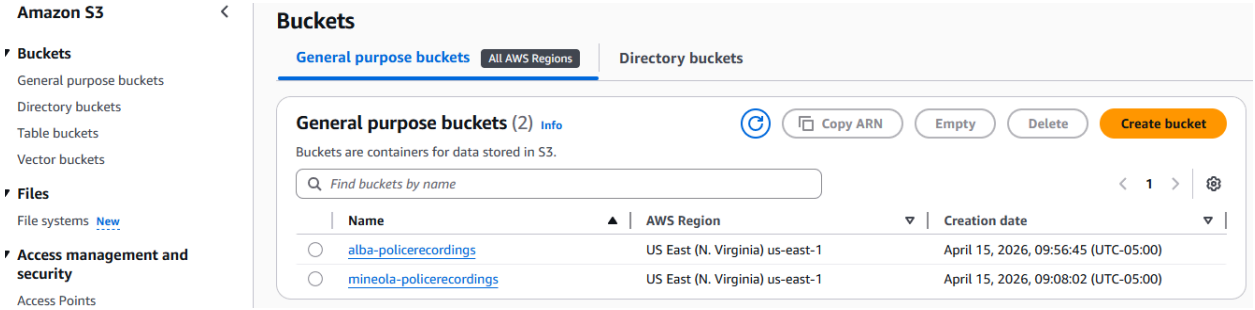
```
{
  "recordingOptions": {
    "destinationUrl": "s3://mineola-policerecordings/mineola-dispatch1recordings/",
    "credentials": "AKIAQKSCLE75ITVJSD2C:iw5ptqud/wKvgyfgnQ/ceFy/mo019v/yxEa5bfjk:us-east-1",
    "notificationEvents": true
  }
}
```

2. Next, I created a Mineola IAM user to be used by the designated government official to access and download the phone recordings when needed.
3. I gave read permissions only to this IAM user

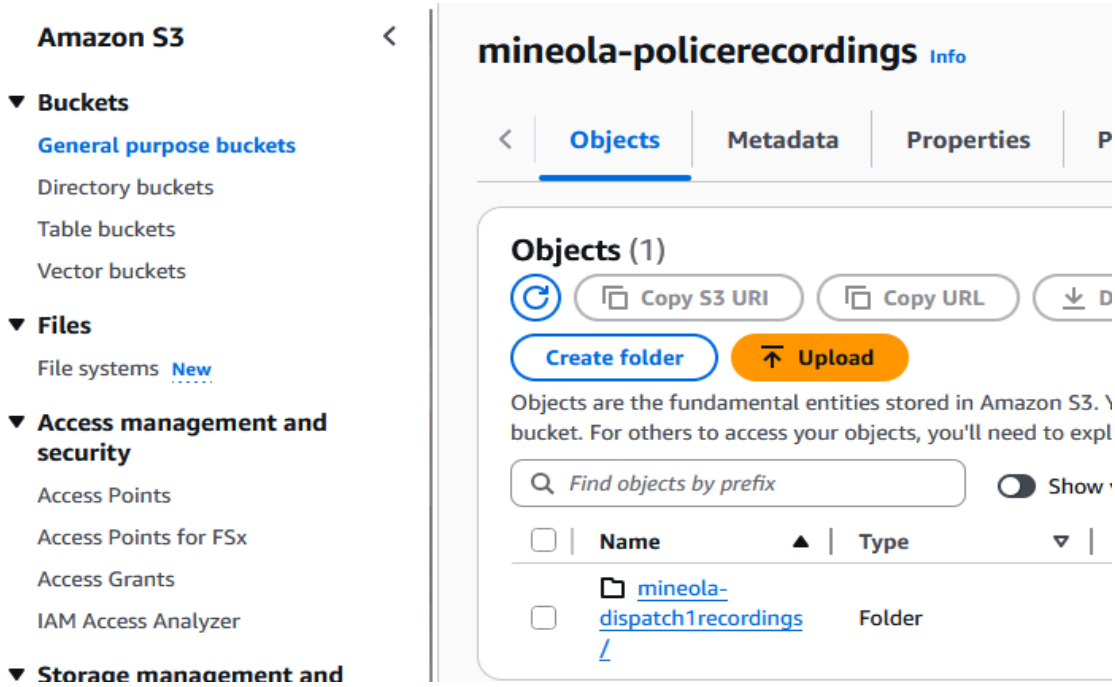


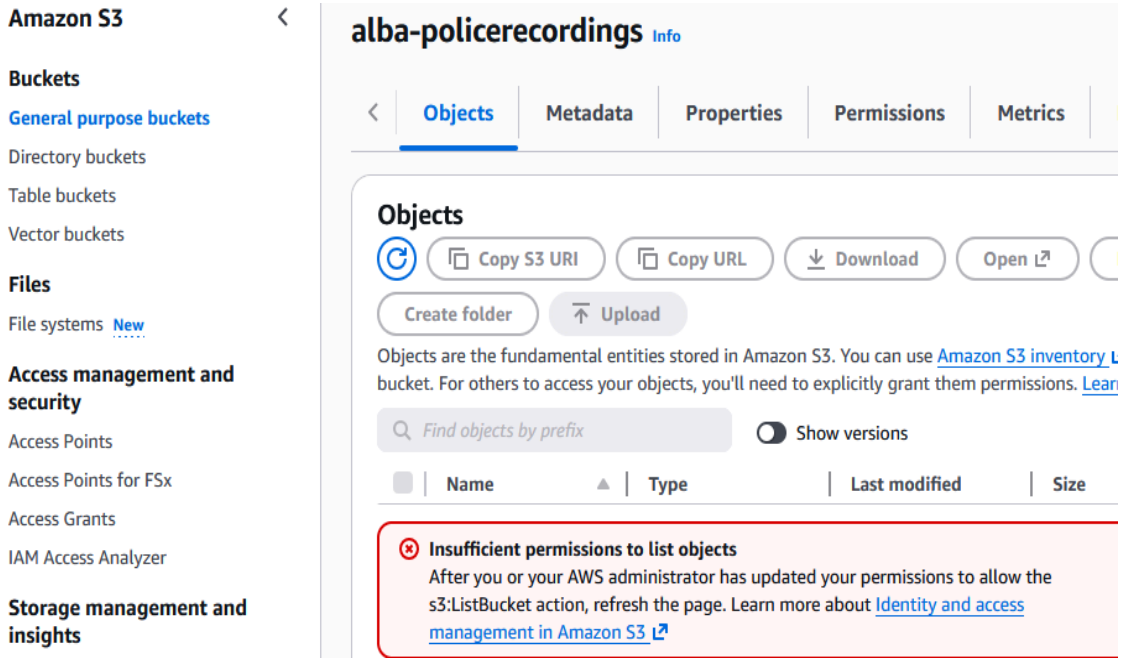
Creating S3 buckets

1. I created two s3 buckets for the files to be uploaded into.
2. I enabled bucket versioning to keep track if any changes were made. This is for compliance purposes



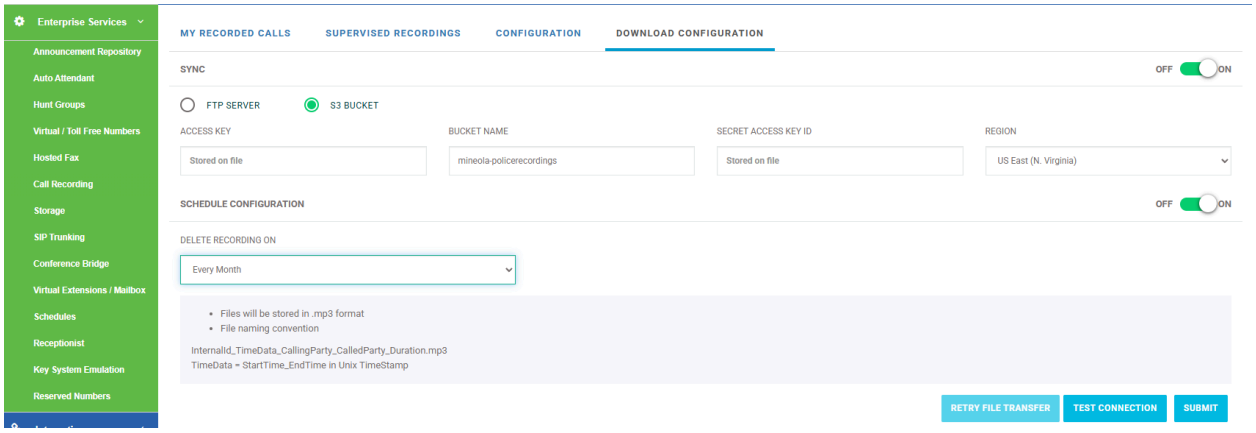
3. I also enabled permissions to allow IAM users access to the buckets based upon the department/organization they are apart of.
4. I tested my configurations by logging in as the Mineola IAM user and trying to access both the Mineola and Alba folder. I was only able to access files in the Mineola folder confirming my ACLs worked



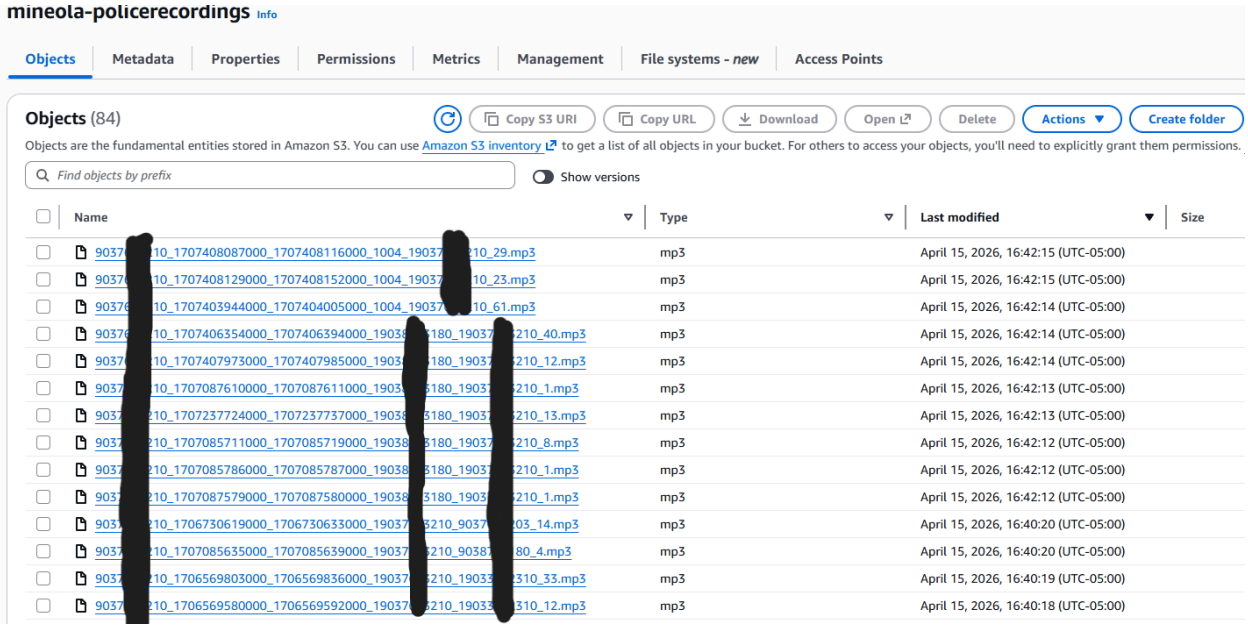


Configure VIP/Sinch Recording Database

1. Logged into the VIP online portal and went to **call recording** tab on left
2. Entered in the Access key and Secret key for the Sinch IAM account I got earlier



3. After a few minutes, the files started to appear in the S3 bucket



4. The files were all loaded into one bucket with no organization.
5. I wrote a Lambda function that would organize the files based on the first 10 characters of the file (the phone number that initiated the call. It then placed files into separate folders based upon their name

```
import boto3
import re
import urllib.parse

s3 = boto3.client('s3')

BUCKET_NAME = "mineola-policerecordings"

def lambda_handler(event, context):
    # If triggered by S3 event (new uploads)
    if 'Records' in event:
        for record in event['Records']:
            key = urllib.parse.unquote_plus(record['s3']['object']['key'])
            process_file(key)
    else:
        # Manual invocation: process EVERYTHING already in bucket
        process_all_files()

def process_all_files():
    paginator = s3.get_paginator('list_objects_v2')

    for page in paginator.paginate(Bucket=BUCKET_NAME):
        for obj in page.get('Contents', []):
            key = obj['Key']
            process_file(key)

def process_file(key):
    # Skip "folders" and already-organized files
    if key.endswith('/') or is_already_sorted(key):
        return

    filename = key.split('/')[-1]

    # Extract first 10 digits from filename
    match = re.match(r'^(\d{10})', filename)

    if not match:
        print(f"Skipping {filename} (no 10-digit prefix)")
        return

    folder = match.group(1)
    new_key = f"{folder}/{filename}"

    # Avoid unnecessary copy if already correct
    if key == new_key:
        return

    print(f"Moving {key} -> {new_key}")
```

```
    # Copy to new location
    s3.copy_object(
        Bucket=BUCKET_NAME,
        CopySource={'Bucket': BUCKET_NAME, 'Key': key},
        Key=new_key
    )

    # Delete original
    s3.delete_object(
        Bucket=BUCKET_NAME,
        Key=key
    )

def is_already_sorted(key):
    """
    Check if file is already inside a folder that matches its prefix
    Example: 1234567890/file.wav
    """
    parts = key.split('/')

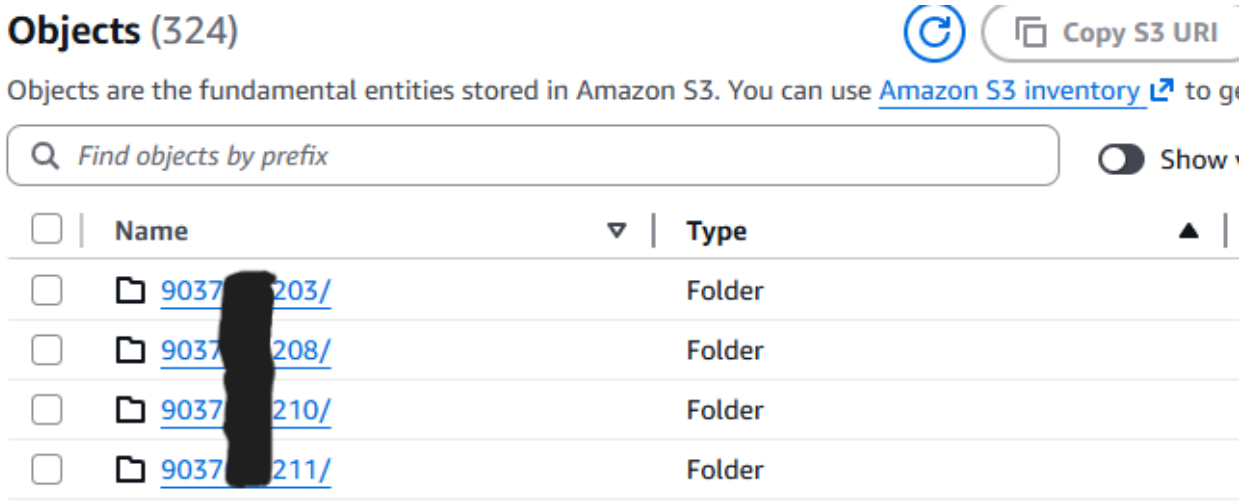
    if len(parts) < 2:
        return False

    folder = parts[0]
    filename = parts[-1]

    match = re.match(r'^(\d{10})', filename)
    if not match:
        return True  # treat as "do nothing"

    return folder == match.group(1)
```

6. I ran the script and it started automatically creating folders and pulling files into the folders based upon outbound caller



This project really helped me understand how AWS and third party apps work together. The hardest part was writing the Lambda script and also settings different permissions for the 3 IAM accounts (Sinch, Mineola, and Lambda function IAM role)